

1 · Introduction to solving ODEs with DifferentialEquations.jl ⇄

This notebook is a first tour of the SciML differential-equation solvers, from a plain exponential decay to a chaotic system, ending with an interactive damping slider.

```
1 using OrdinaryDiffEq, Plots, PlutoUI
```

1. Exponential decay ⇄

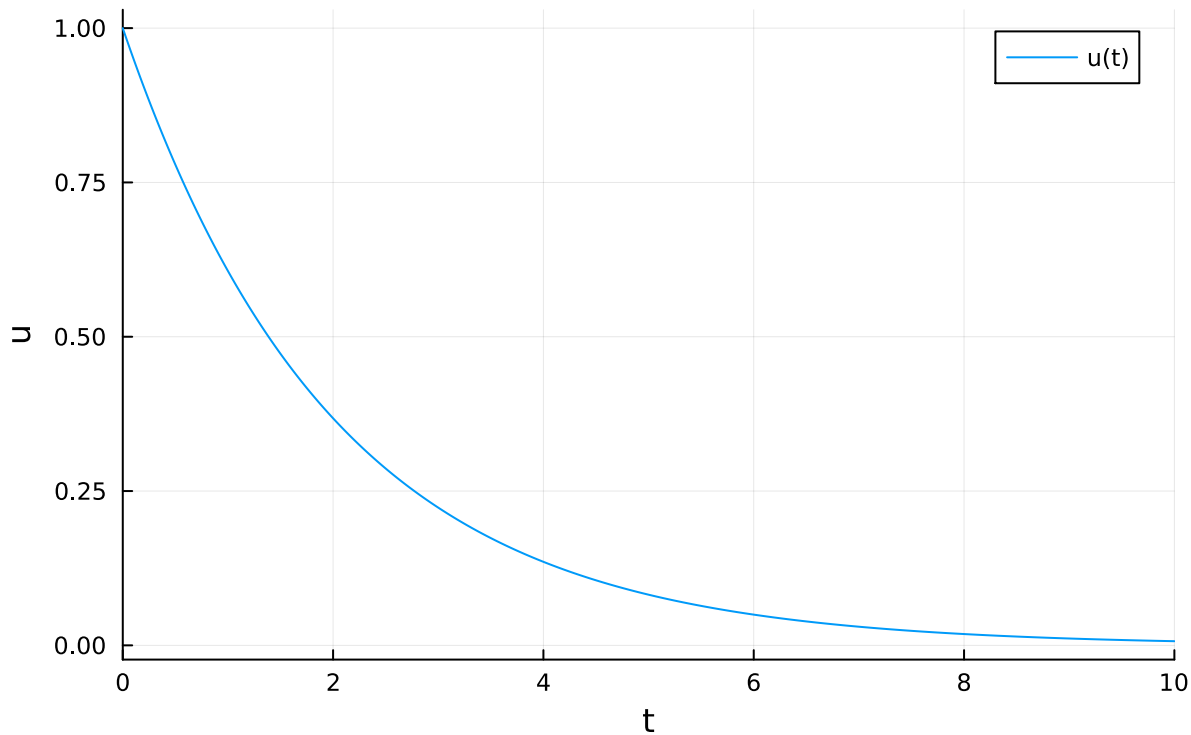
The simplest possible ODE: $\dot{u} = \lambda u$.

```
► SciMLBase.ODESolution{Float64, 1, Vector{Float64}, Nothing, Nothing, Vector{Float64}, Vector{Vect
```

```
◀ ██████████ ▶
```

```
1 begin
2     λ = -0.5
3     u0_decay = 1.0
4     tspan_decay = (0.0, 10.0)
5     decay_prob = ODEProblem((u, p, t) -> λ * u, u0_decay, tspan_decay)
6     decay_sol = solve(decay_prob, Tsit5())
7 end
```

Exponential decay $du/dt = \lambda u$



```
1 plot(decay_sol; label="u(t)", xlabel="t", ylabel="u", title="Exponential decay du/dt =  
λu")
```

2. A nonlinear system: the damped pendulum $\Leftrightarrow$

$$\dot{\theta} = \omega, \quad \dot{\omega} = -\frac{g}{L} \sin \theta - \gamma \omega$$

pendulum! (generic function with 1 method)

```
1 function pendulum!(du, u, p, t)  
2     θ, ω = u  
3     g, L, γ = p  
4     du[1] = ω  
5     du[2] = -(g / L) * sin(θ) - γ * ω  
6     return nothing  
7 end
```

```

retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 83-element Vector{Float64}:
 0.0
 0.00011759418272624786
 0.0037290043759967686
 0.030155564132355806
 0.09443343501296676
 0.19389410719712286
 0.3247412439831845
  ⋮
18.774693120007427
19.056140770916926
19.314406392402407
19.58960194289681
19.85458557380189
20.0
u: 83-element Vector{Vector{Float64}}:
 [1.0471975511965976, 0.0]
 [1.0471974924562908, -0.0009990283479762883]
 [1.0471385051224469, -0.03166246295849604]
 [1.0433477850063995, -0.25484761264334366]
 [1.0098098671901836, -0.7851557402119058]
 [0.8930610380658826, -1.547713263446728]
 [0.6342215010068161, -2.363478617254933]
  ⋮
 [0.00781808695009879, -0.19031191447829365]
 [-0.03992141157131463, -0.1275470798953339]
 [-0.05622528538891211, 0.006483888779814289]
 [-0.0356204416923234, 0.13210717998742313]
 [0.005583307187236967, 0.16042275215504803]
 [0.02706768048955084, 0.1301651037466124]

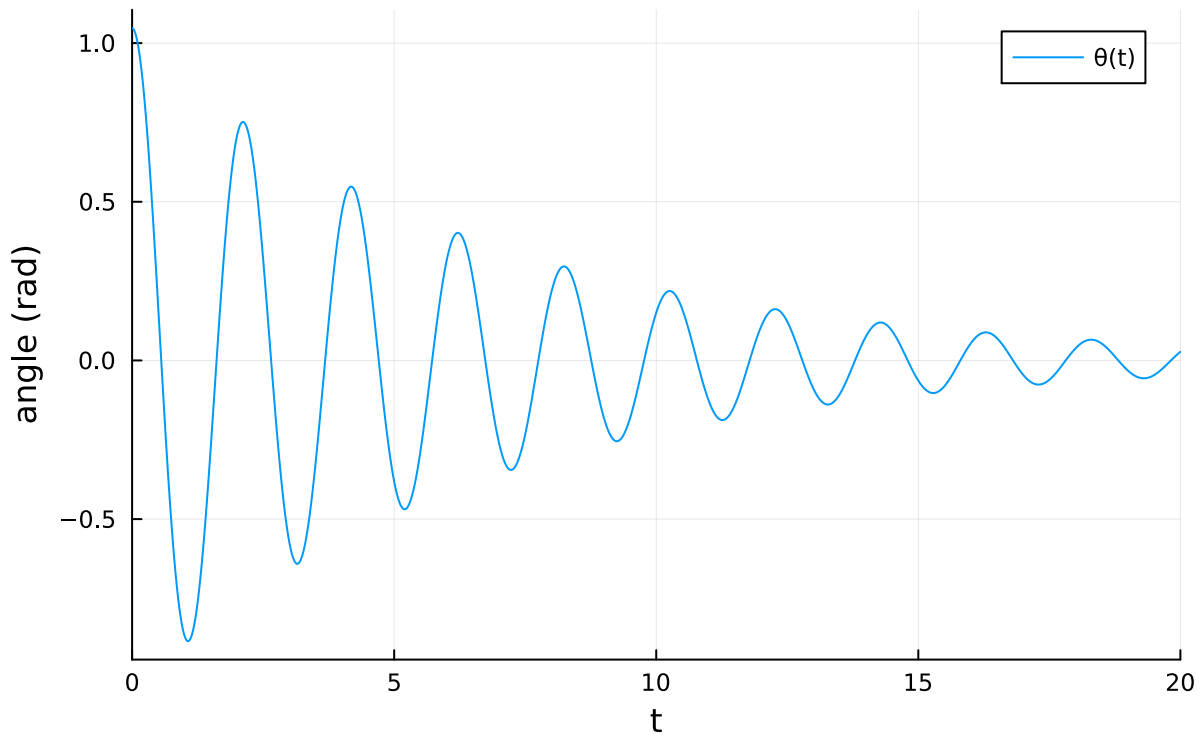
```

```

1 begin
2     p_pend = (9.81, 1.0, 0.3) # g, L, γ
3     u0_pend = [π / 3, 0.0]    # initial angle, angular velocity
4     tspan_pend = (0.0, 20.0)
5     pend_prob = ODEProblem(pendulum!, u0_pend, tspan_pend, p_pend)
6     pend_sol = solve(pend_prob, Tsit5())
7 end

```

Damped pendulum



```
1 plot(pend_sol; idxs=1, label="θ(t)", xlabel="t", ylabel="angle (rad)", title="Damped pendulum")
```

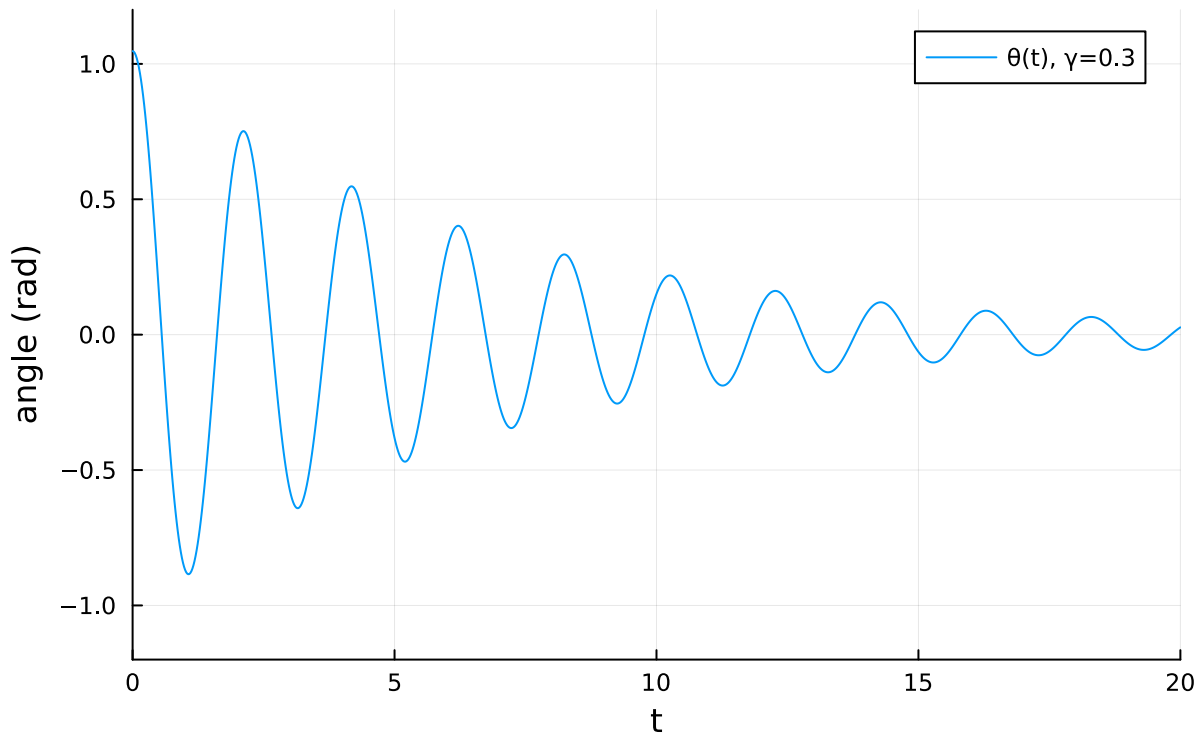
3. Interactivity: drag the damping coefficient ⇌

Pluto reruns every dependent cell automatically whenever `γ_interactive` changes.

0.3

```
1 @bind γ_interactive Slider(0.0:0.05:2.0, default=0.3, show_value=true)
```

Damping slider demo



```
1 begin
2   p_interactive = (9.81, 1.0, y_interactive)
3   interactive_prob = ODEProblem(pendulum!, u0_pend, tspan_pend, p_interactive)
4   interactive_sol = solve(interactive_prob, Tsit5())
5   plot(interactive_sol; idxs=1, label="θ(t)", γ=$(y_interactive), xlabel="t",
6        ylabel="angle (rad)",
7        title="Damping slider demo", ylims=(-1.2, 1.2))
8 end
```

4. A taste of chaos: the Lorenz attractor ⇄

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z$$

lorenz! (generic function with 1 method)

```
1 function lorenz!(du, u, p, t)
2   σ, ρ, β = p
3   x, y, z = u
4   du[1] = σ * (y - x)
5   du[2] = x * (ρ - z) - y
6   du[3] = x * y - β * z
7   return nothing
8 end
```

```

retcode: Success
Interpolation: 1st order linear
t: 4001-element Vector{Float64}:
 0.0
 0.01
 0.02
 0.03
 0.04
 0.05
 0.06
 ⋮
39.95
39.96
39.97
39.98
39.99
40.0
u: 4001-element Vector{Vector{Float64}}:
 [1.0, 0.0, 0.0]
 [0.9179244496885618, 0.26633998668091097, 0.0012638983207408072]
 [0.8679194496774282, 0.5117404281748704, 0.00465546057209661]
 [0.8453601465107559, 0.7446541555633573, 0.009835884893227667]
 [0.8468051748218214, 0.9723329094327556, 0.016734522370772373]
 [0.8697867489622433, 1.201131649024005, 0.025486210592314894]
 [0.912640019808693, 1.4367682249279918, 0.03640123169101509]
 ⋮
 [8.94987468002127, 14.53953019337956, 17.99128012273739]
 [9.519613675276286, 15.274884005461443, 18.879257460124073]
 [10.102884058095956, 15.964362322653129, 19.895437745748325]
 [10.692382754622283, 16.585185569853365, 21.041253638189726]
 [11.279591539420966, 17.111655880870487, 22.315359654642332]
 [11.855224407107182, 17.515967885574153, 23.706492586162582]

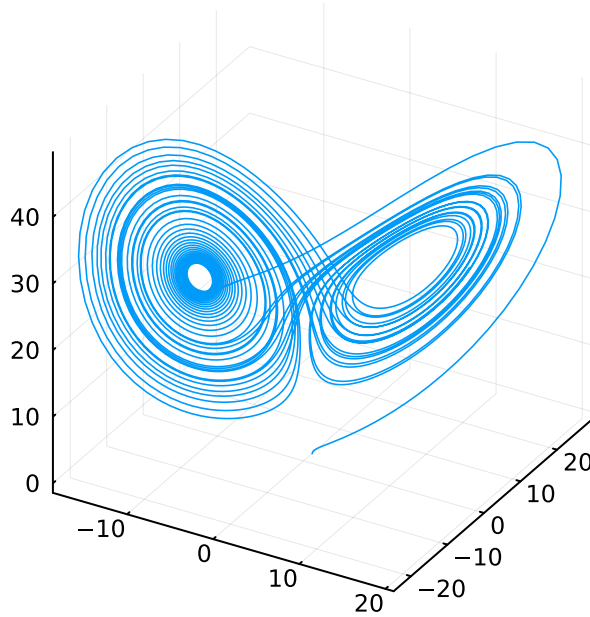
```

```

1 begin
2     lorenz_p = (10.0, 28.0, 8 / 3)
3     lorenz_u0 = [1.0, 0.0, 0.0]
4     lorenz_tspan = (0.0, 40.0)
5     lorenz_prob = ODEProblem(lorenz!, lorenz_u0, lorenz_tspan, lorenz_p)
6     lorenz_sol = solve(lorenz_prob, Tsit5(); saveat=0.01)
7 end

```

Lorenz attractor



```
1 plot(lorenz_sol; idxs=(1, 2, 3), camera=(30, 30), label=false, title="Lorenz attractor", linewidth=0.8)
```

Takeaways ⇌

- `ODEProblem(f, u0, tspan, p)` plus `solve(prob, alg)` is all you need for scalar equations, systems, and chaotic systems alike.
- Swapping the algorithm (`Tsit5()`, `Rodas5()`, ...) or tolerances is a one-line change — the modeling code never has to change.
- `@bind` from `PlutoUI` turns any notebook into a live, explorable simulation.

Next: `02_neural_ode.jl` replaces part of the right-hand side of an ODE with a neural network.